Ders 3 Çalışma Özeti

Ders 3 Çalışma Özeti

Genel Konular

- Grupların yeniden dağıtılması ve ödev takvimi.
- Bağlantı problemi (connectivity problem) ve dizi tabanlı, küme kimliği tutan çözüm (quick-find).
- Neden verimli algoritma tasarlarız: 600 bin kelimede sıralı arama ile ikili arama karşılaştırması.
- Algoritma tasarım teknikleri: kaba kuvvet (brute force), açgözlü (greedy), böl ve yönet (divide and conquer), azalt ve yönet (decrease and conquer), dönüştür ve yönet (transform and conquer), dinamik programlama (dynamic programming), yinelemeli iyileştirme (iterative improvement), geri izleme (backtracking).
- Ortak bölenlerin en büyüğü (OBEB, greatest common divisor): asal çarpan yöntemi ve Öklid algoritması.
- Eratosthenes Kalburu'nun tekrarı ve 0/1 dizisiyle alternatif akış.
- Algoritma analizi: deneysel (empirical) ve teorik analiz.
- İşlem maliyetlerinin sayılması: ardışık komutlar, `if`, tek döngü, iç içe döngüler.
- Büyüme oranı (growth rate) ve yaygın büyüme fonksiyonları.
- Big O, Big Omega ve Big Theta gösterimleri; Big O'nun özellikleri.
- En kötü, en iyi ve ortalama durum analizi (worst/best/average case).
- Sıralı arama (sequential search), ikili arama (binary search), bubble sort, selection sort ve insertion sort'un karmaşıklıkları.

Hocanın Özellikle Vurguladığı Kısımlar

- Algoritma tasarımında doğru veri yapısını seçmek çözümü çok basitleştirir. Bağlantı problemi tek bir diziyle çözülebilir.
- Problem çok zorsa, işe yarar bilgiyi veren daha basit bir probleme dönüştürülebilir. Örneğin "hangi yolla bağlı?" yerine yalnız "bağlı mı?" sorusu cevaplanır.
- Coding challenge sitelerindeki problemleri önce kendiniz çözün, sonra hazır çözümlerle karşılaştırın.
- Bir algoritmanın her adımı açık ve tanımlı olmalı. "Asal çarpanlarını bul" diyen ama nasıl bulunacağını söylemeyen bir yöntem algoritma sayılmaz.
- Çalışma süresini kronometreyle ölçmek, algoritmayı değil gerçekleştirmeyi (implementation) karşılaştırır. Bu yüzden teorik analiz tercih edilir.
- Bu derste algoritmalar hep en kötü durum üzerinden ve Big O ile, yani üstten sınırlanarak incelenecek.
- Big O'da düşük dereceli terimler ve sabit çarpanlar atılır. Başka bir değişkene bağlı çarpan (örneğin m, ya da graflarda V ve E) atılmaz.
- Logaritmalar bu derste hep 2 tabanındadır.
- Dersi tekrar izleyin. Önceden yazdığınız algoritmaların en kötü, en iyi ve ortalama durumlarını kendiniz analiz edin. Akış diyagramlarını mutlaka çizin.

Kısa Tekrar Notları

- **Bağlantı problemi:** Çiftler sırayla gelir. Her çift için "bu ikisi zaten bağlı mı?" sorusu cevaplanır. Bağlı değillerse kümeleri birleştirilir.

- **Quick-find:** `id[i]` dizisi her düğümün küme kimliğini tutar. Aynı kimlik, aynı bağlı bileşen (connected component) demektir.
- **Öklid algoritması:** $\gcd(m, n) = \gcd(n, m \bmod n)$; $n = 0$ olunca sonuç m'dir.
- **Big O:** $0 \leq f(n) \leq k \cdot g(n)$, her $n \geq n_0$ için. Üst sınır.
- **Big Omega:** $f(n) \geq k \cdot g(n)$, her $n \geq n_0$ için. Alt sınır.
- **Big Theta:** $k_1 g(n) \leq f(n) \leq k_2 g(n)$, her $n \geq n_0$ için. Hem alt hem üst sınır.
- **Büyüme sırası (iyiden kötüye):** 1, $\log n$, $\log^2 n$, n, $n \log n$, n^2, n^3, 2^n, $n!$, n^n.
- **Döngü kuralları:**
  - Tek döngü: tekrar sayısı × gövdenin maliyeti.
  - İç içe döngü: tüm döngülerin tekrar sayılarının çarpımı × en içteki gövdenin maliyeti.
  - Ardışık komutlar: maliyetler toplanır.
  - `if`: karşılaştırma maliyeti + iki koldan pahalı olanın maliyeti.
- **Karmaşıklık özeti:**

Algoritma	En iyi	Ortalama	En kötü
Sıralı arama	$O(1)$	$O(n)$	$O(n)$
İkili arama	$O(1)$	$O(\log n)$	$O(\log n)$
Bubble sort (bayraksız)	$O(n^2)$	$O(n^2)$	$O(n^2)$
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$

Detaylı Açıklamalar

Bağlantı problemi

- Girdi, sırayla gelen p-q çiftleridir. p-q, "p ile q bağlı" demektir. Bağlantı yönsüzdür: p q'ya bağlıysa q da p'ye bağlıdır.
- Amaç, gelen her çiftin yeni bir bağlantı mı olduğunu, yoksa zaten dolaylı olarak bağlı mı olduğunu bulmaktır.
- Örnek: 3-4, 4-9, 8-0, 2-3, 5-6 geldikten sonra 2-9 gelirse, 2-3-4-9 yolu zaten var. Yeni bağlantı kurulmaz.
- **Uygulama alanları:** bilgisayar ağındaki düğümler, baskılı devredeki kontaklar, sosyal ağda arkadaşın arkadaşı ilişkileri, programdaki değişken adları.
- Bir öğrenci matris ve yol arama algoritması önerdi. Çalışır ama her sorguda ayrı bir algoritma koşturmak gerekir; verimli değildir.
- n nesneyi bağlamak için $n - 1$ bağlantı yeterlidir. Bu konu minimum yayılan ağaçta (minimum spanning tree) tekrar görülecek.

Dizi ile çözüm (quick-find):

- Başlangıçta her düğüm kendi kümesindedir: `id[i] = i`.
- `p-q` geldiğinde:
  - `id[p] == id[q]` ise zaten bağlıdırlar, hiçbir şey yapılmaz.
  - Değilse, kimliği `id[p]` olan bütün elemanlar `id[q]` yapılır.
- 10 düğümlü örneğin izlenmesi:

Gelen çift	0	1	2	3	4	5	6	7	8	9
Başlangıç	0	1	2	3	4	5	6	7	8	9
4-3	0	1	2	3	3	5	6	7	8	9
3-8	0	1	2	8	8	5	6	7	8	9
6-5	0	1	2	8	8	5	5	7	8	9
9-4	0	1	2	8	8	5	5	7	8	8
2-1	0	1	1	8	8	5	5	7	8	8
8-9	zaten bağlı, değişiklik yok									
5-0	0	1	1	8	8	0	0	7	8	8
7-2	0	1	1	8	8	0	0	1	8	8
6-1	1	1	1	8	8	1	1	1	8	8

- Sonuçta iki bileşen kalır: {0, 1, 2, 5, 6, 7} (kimlik 1) ve {3, 4, 8, 9} (kimlik 8).
- "1 ile 8 bağlı mı?" sorusu: `id[1] = 1`, `id[8] = 8`, farklı. Bağlı değiller.

Neden verimli algoritma?

- 3000 mektup, her birinde 20 satır ve satır başına 10 kelime var. Toplam 600 bin kelime, alfabetik sıralı.
- Her kelime karşılaştırması 1 saniye sürsün. Bir kelimenin kaç kez geçtiği aranıyor.
- **Sıralı arama:** 600 bin karşılaştırma, yaklaşık 166 saat (neredeyse 1 hafta).
- **İkili arama:** $\log_2 600000 \approx 20$ karşılaştırma, yaklaşık 20 saniye.
- Girdinin sıralı olduğu bilgisi kullanılarak (exploit) çok daha hızlı bir algoritma seçilebilir.

Algoritma tasarım teknikleri

- **Kaba kuvvet (brute force):** Bütün olasılıkları dener. Örnek: şifresi unutulmuş RAR dosyasında tüm şifreleri denemek.
- **Açgözlü (greedy):** Her adımda o an en iyi görünen seçimi yapar. Örnek: 1, 5 ve 3 elma olan üç kapıdan 5 elmalı olanı seçmek.
- **Böl ve yönet (divide and conquer):** Problemi parçalara böler, parçaları çözer, sonra birleştirir. Örnek: merge sort.
- **Azalt ve yönet (decrease and conquer):** Arama uzayını adım adım küçültür. Örnek: insertion sort.
- **Dönüştür ve yönet (transform and conquer):** Problemi, çoğunlukla bir veri yapısı kullanarak, başka bir biçime dönüştürüp çözer.
- **Dinamik programlama:** Aslında bir optimizasyon tekniğidir. Örnek: üstel karmaşıklıklı Fibonacci hesabını doğrusal hâle getirmek.
- **Yinelemeli iyileştirme (iterative improvement):** Örnek: Newton-Raphson ile kök bulma.
- **Geri izleme (backtracking):** Örnek: sudoku çözme.
- Bunların ayrıntısı 3. sınıftaki Algoritma Analizi dersinde görülecek.

OBEB ve Öklid algoritması

Ortaokul yöntemi:

- m'yi ve n'yi asal çarpanlarına ayır. Ortak asal çarpanları çarp.
- Örnek: $6 = 2 \cdot 3$, $18 = 2 \cdot 3^2$. Ortak çarpanlar 2 ve 3, OBEB = 6.
- 1 asal sayı değildir; asal sayılar 2'den başlar.
- Bu yöntem bu hâliyle algoritma değildir. "Asal çarpanlarını bul" adımının nasıl yapılacağı tanımlanmamıştır.

Öklid algoritması (Elements, MÖ 300 civarı):

- Kural: $\gcd(m, n) = \gcd(n, m \bmod n)$. İşlem $n = 0$ olana kadar tekrarlanır, sonuç m'dir.
- Örnek: $\gcd(60, 24) = \gcd(24, 12) = \gcd(12, 0) = 12$.
- Örnek: $\gcd(1785, 546)$:

m	n	$m \bmod n$
1785	546	147
546	147	105
147	105	42
105	42	21
42	21	0
21	0	sonuç: 21

- Sözde kod:

```
Adım 1: n = 0 ise m'yi döndür ve dur; değilse Adım 2'ye geç.
Adım 2: m'yi n'ye böl, kalanı r'ye ata.
Adım 3: m ← n, n ← r. Adım 1'e dön.
```

- Akış: `m, n` okunur. `n ≠ 0` olduğu sürece (`while`): `r = m mod n`, `m = n`, `n = r`. Döngüden çıkınca `m` yazdırılır.
- Algoritmanın doğruluğunu göstermek için n'nin bir gün mutlaka 0 olacağını ispatlamak gerekir. Hoca bunu düşünmeyi ödev olarak bıraktı.

Eratosthenes Kalburu'nun tekrarı

- 1'den 36'ya kadar örnek: $\sqrt{36} = 6$'ya kadar gitmek yeter.
- 2'nin, 3'ün ve 5'in katları silinir. 4 ve 6 zaten silinmiştir.
- Kalanlar: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31.
- **Alternatif gösterim:** Sayıları ayrıca saklamak yerine, dizi indisi sayıyı temsil eder. Dizide yalnız 1 (asal) ya da 0 (asal değil) tutulur.
  - Başta hepsi 1'dir. Silinmemiş her i için $i \cdot j \leq n$ oldukça `A[i * j] = 0` yapılır.
  - Gömülü sistemlerde bu değerler bit olarak saklanırsa çok büyük yer kazancı olur.

Algoritma analizi

- Amaç: farklı çözümlerin başarımını (performance) ölçmek, karşılaştırmak ve daha iyisinin mümkün olup olmadığını görmek.

Deneysel (empirical) analiz:

- Algoritmalar kodlanır, çalıştırılır ve süreleri ölçülür.
- Sakıncaları:
  - Algoritmayı değil, kodlamayı karşılaştırır. Programcının becerisi sonucu etkiler.
  - Bilgisayar, o an çalışan diğer süreçler, sayfalama (paging), ağ trafiği ve derleyici optimizasyonları sonucu etkiler.
- Çok sayıda deney yapıp ortalama almak etkiyi azaltır ama yine iyi bir yöntem değildir.

Teorik analiz:

- Gerçekleştirmeden, ortamdan ve veriden bağımsızdır.
- Önce önemli işlemlerin sayısı (maliyet noktaları) belirlenir. Sonra girdi büyüdükçe bu sayının nasıl büyüdüğü, büyüme fonksiyonlarıyla ifade edilir.

İşlem maliyetlerinin sayılması

- Her işlemin (atama, aritmetik, karşılaştırma, döngü kontrolü) işlemciye bağlı sabit bir maliyeti vardır.
- **Ardışık komutlar:** `count = count + 1;` (c1) ve `sum = sum + count;` (c2). Toplam: $c_1 + c_2$.
- **`if`:** Karşılaştırma her zaman yapılır. İki koldan yalnız biri çalışır. Toplam: $c_1 + \max(c_2, c_3)$.
- **Tek döngü:**

Satır	Maliyet	Tekrar
`i = 1;`	c1	1
`sum = 0;`	c2	1
`while (i <= n)`	c3	$n + 1$
`i = i + 1;`	c4	n
`sum = sum + i;`	c5	n

- Toplam: $c_1 + c_2 + (n+1)c_3 + nc_4 + nc_5 = an + b$. Süre n ile doğru orantılıdır.
- **İç içe iki döngü:** İç döngü kontrolü $n(n+1)$ kez, iç gövde $n \cdot n$ kez çalışır. Toplam $an^2 + bn + c$ biçimindedir, n^2 ile orantılıdır.
- İç içe üç döngü n^3 mertebesine çıkar.

Büyüme oranı

- Girdi boyutu n büyüdükçe çalışma süresinin ne kadar hızlı arttığıdır.
- Örnek: A algoritması $n^2/5$, B algoritması $5n$ birim zaman istiyor. A kare ile, B doğrusal büyür.

- Sayısal örnek: Doğrusal algoritma $n = 10$ için 0,01 s, $n = 1000$ için 1 s sürüyor. Karesel algoritma $n = 1000$'de onunla başa baş geliyor.

n	Doğrusal	Karesel
1000	1 s	1 s
10 000	10 s	100 s
100 000	100 s	10 000 s (yaklaşık 2,77 saat)

- Başlarda hızlı görünen bir algoritma, belli bir eşikten sonra çok yavaşlayabilir.

Yaygın büyüme oranları:

Gösterim	Ad	Açıklama
$O(1)$	Sabit	Girdi boyutundan bağımsız
$O(\log n)$	Logaritmik	İkiye bölerek ilerleyen algoritmalar, örneğin ikili arama
$O(\log^2 n)$	Log-kare	
$O(n)$	Doğrusal	
$O(n \log n)$	Doğrusal logaritmik (linearithmic)	
$O(n^2)$	Karesel (quadratic)	
$O(n^3)$	Kübik	
$O(2^n)$	Üstel (exponential)	Örneğin özyinelemeli Fibonacci
$O(n!)$	Faktöriyel	Bütün permütasyonları denemek (exhaustive search)
$O(n^n)$		Faktöriyelden de kötü

- Girdi boyutuna göre işlem sayıları:

n	$\log n$	$n \log n$	n^2	n^3
10	3	30	10^2	10^3
100	6	600	10^4	10^6
1000	9	9000	10^6	10^9
10 000	13	130 000	10^8	10^{12}
100 000	16	$1,6 \cdot 10^6$	10^{10}	10^{15}
1 000 000	19	$\approx 10^7$	10^{12}	10^{18}

- 2^n için $n = 100$'de yaklaşık 10^{30}, $n = 1000$'de yaklaşık 10^{301} işlem gerekir. Teoride çözer ama pratikte uygulanamaz (feasible değildir).
- **İkili aramanın gücü:** 16 elemanda 4, 16 bin elemanda 14, 1 milyon elemanda 20, 1 milyar elemanda 30 adım.

Döngü örnekleri:

```
for (i = 0; i < n; i++)          /* n adım: O(n) */
for (i = 0; i < n; i += 2)       /* n/2 adım: yine O(n) */
for (i = 1; i < n; i *= 2)       /* log n adım: O(log n); n = 1000 için 1, 2, 4, ..., 512 = 10

for (i = 0; i < n; i++)          /* n * n: O(n^2) */
    for (j = 0; j < n; j++) ...

for (i = 0; i < n; i++)          /* bağımlı karesel: n(n+1)/2, yine O(n^2) */
    for (j = 0; j < i; j++) ...
```

```
for (i = 0; i < n; i++)          /* n * log n: O(n log n) */
    for (j = 1; j < n; j *= 2) ...
```

Big O, Big Omega, Big Theta

- **Big O (üst sınır):** $f(n) = O(g(n))$, eğer pozitif k ve n_0 sabitleri için $0 \leq f(n) \leq k \cdot g(n)$, her $n \geq n_0$.
  - Örnek: $f(n) = n^2 - 3n + 10$. $k = 3$ ve $n_0 = 2$ için $3n^2 \geq n^2 - 3n + 10$, her $n \geq 2$. Yani $f(n) = O(n^2)$.
  - n_0'dan önceki bölge önemsizdir. Sonrasında f, $k \cdot g$'yi asla geçemez.
- **Big Omega (alt sınır):** $f(n) \geq k \cdot g(n)$, her $n \geq n_0$. $f(n) = \Omega(g(n))$.
- **Big Theta (iki taraflı sınır):** $k_1 g(n) \leq f(n) \leq k_2 g(n)$, her $n \geq n_0$. $f(n) = \Theta(g(n))$.
- Eşitlik yerine kesin eşitsizlik kullanılan sürümler küçük o ve küçük omega'dır. Küçük teta yoktur.
- Örnek: $f(n) = n(n+1)/2 = n^2/2 + n/2$, $O(n^2)$'dir.
- "Big O" yazılışındaki harf sıfırla karışmasın diye O harfidir.

Big O'nun özellikleri:

- Düşük dereceli terimler atılır: $n^3 + 4n^2 + 3n = O(n^3)$.
- Sabit çarpanlar atılır: $5n^3 = O(n^3)$.
- Başka bir girdiye bağlı çarpan atılmaz: mn^3 olarak kalır.
- Toplama: $O(f(n)) + O(g(n)) = O(f(n) + g(n))$. Örneğin $O(n^3) + O(4n^2) = O(n^3)$.
- Çarpma da benzer biçimde birleştirilir; iç içe yapılarda dereceler artar.
- Örnekler:
  - Tek döngü: $an + b$, $O(n)$.
  - İki iç içe döngü: $an^2 + bn + c$, $O(n^2)$.
  - `i = 1..n`, `j = 1..i`, `k = 1..j` üçlü döngü: $O(n^3)$.

En kötü, en iyi ve ortalama durum

- Aynı boyuttaki farklı girdiler farklı süre isteyebilir.
- Örnek dizi: 2, 3, 6, 7, 9, 10, 11 ($n = 7$).
- **En kötü durum (worst case):** Algoritmanın n boyutlu girdi için ihtiyaç duyduğu en uzun süre. Örneğin 15 aranırsa 7 karşılaştırmanın hepsi yapılır.
- **En iyi durum (best case):** En kısa süre. Örneğin 2 aranırsa tek karşılaştırmada bulunur. Çok kullanışlı değildir ama "bundan azı olamaz" bilgisini verir.
- **Ortalama durum (average case):** Ortalama süre. Belirlemek zordur; olası girdilerin dağılımına bakılır. Örneğin 210 rastgele arama yapılıp adım sayılarının ortalaması alınır.
- Bu derste hep en kötü durum ve Big O kullanılacak.

Sıralı arama

```
int sequentialSearch(int a[], int size, int x) {
    int i;
    for (i = 0; i < size && a[i] != x; i++);
    if (i == size)
        return -1;
    return i;
}
```

- Bulunamazsa: $O(n)$.
- En iyi durum (aranan ilk eleman): $O(1)$.
- En kötü durum (aranan son eleman): $O(n)$.
- Ortalama: Her konum eşit olasılıklıysa yaklaşık $n/2$ karşılaştırma. Sabit atılır, $O(n)$.

İkili arama

```
int binarySearch(int a[], int size, int x) {
    int low = 0;
```

```
    int high = size - 1;
    int mid; /* aranan değer bulunursa mid konumu tutar */
    while (low <= high) {
        mid = (low + high) / 2;
        if (a[mid] < x)
            low = mid + 1;
        else if (a[mid] > x)
            high = mid - 1;
        else
            return mid;
    }
    return -1;
}
```

- 2, 3, 6, 7, 9, 10, 11 dizisinde:
  - 7 aranırsa ortadaki elemanda, ilk adımda bulunur. En iyi durum $O(1)$.
  - 15 aranırsa 7, 10, 11 ile karşılaştırılır, sonra arama aralığı boşalır. En kötü durum $\log n + 1$ karşılaştırma, yani $O(\log n)$.
- **Ortalama:** 1 eleman 1 adımda, 2 eleman 2 adımda, 4 eleman 3 adımda bulunur. Toplam $1 + 4 + 12 = 17$ karşılaştırma. $17/7 \approx 2{,}43$, $\log_2 7 \approx 2{,}81$'den küçük. Ortalama da $O(\log n)$.
- Her adımda arama uzayının yarısı atıldığı için logaritmiktir.

Sıralama algoritmaları

Bubble sort: Komşu elemanlar karşılaştırılır, büyük olan sona taşınır. En büyük eleman kabarcık gibi sona çıkar.

```
for i ← 0 to n − 2 do
    for j ← 0 to n − 2 − i do
        if A[j + 1] < A[j]
            tmp ← A[j]
            A[j] ← A[j + 1]
            A[j + 1] ← tmp
```

- İç içe iki döngü ve sabit maliyetli bir `if`: $O(n^2)$.

Selection sort: Her turda kalan elemanların en küçüğü seçilir ve sıradaki yere yerleştirilir.

- Karşılaştırma sayısı yaklaşık $n^2/2$, yer değiştirme sayısı n mertebesindedir. Toplam $O(n^2)$.

Insertion sort: İskambil kâğıtlarını elde sıralamaya benzer. Sıradaki eleman alınır, önceki sıralı kısımla karşılaştırılarak doğru yere kaydırılır.

```
for i ← 1 to n-1 do
    key ← A[i]
    // A[i]'yi sıralı A[0..i-1] dizisine ekle
    j ← i - 1
    while (j >= 0 and A[j] > key) do
        A[j+1] ← A[j]
        j ← j - 1
    A[j+1] ← key
```

- En kötü durum: İç `while` her seferinde sona kadar döner, $O(n^2)$.
- En iyi durum (dizi zaten sıralı): İç `while` hiç çalışmaz, $O(n)$.
- Ortalama durum: $O(n^2)$.

Duyurular

- Gruplar OBS'de yeniden dağıtıldı. Ders günü ve saati değişmedi.
- Slaytlar dersten önce değil, dersten sonra paylaşılacak; hoca slaytları ders öncesine kadar değiştirebiliyor.

- Ödev, proje ve vize haftaları bütün gruplarda aynı.
  - Ödevler 5. ve 6. haftalarda başlayacak.
  - İlk ödevin verilme tarihi muhtemelen 28 Mart civarı.
- Ödev kontrolleri şimdilik uzaktan yapılacak. Nisan'daki YÖK kararına göre değişebilir.
- Ertesi gün (salı) uygulama dersi yok.

Ders sonrası notlar

- Hoca Öklid algoritmasında $m > n$ varsayımının ve gerekirse yer değiştirmenin gerektiğini söyledi. Gerekmez: $m < n$ ise ilk adımda m mod $n = m$ olur ve iki sayı kendiliğinden yer değiştirir.
- $\gcd(1785, 546)$ örneğinde sözlü anlatımda 147 mod $105 = 42$ adımı atlandı. Tam adımlar yukarıdaki tabloda.
- Öklid algoritmasının sonlanma ispatı: Her adımda yeni n değeri m mod $n < n$ olur. Negatif olmayan tam sayılar sonsuza kadar azalamaz, bu yüzden n sonunda 0 olur. $\gcd(m, n) = \gcd(n, m \bmod n)$ eşitliği, m ile n'nin ortak bölenlerinin n ile m mod n'nin ortak bölenleriyle aynı olmasından gelir.
- Hoca n^2'li büyümeye birkaç kez "üstel" dedi. n^2 karesel (quadratic), yani polinomsal büyümedir. Üstel büyüme 2^n gibi fonksiyonlardır.
- $n^2/5$ ile $5n$ fonksiyonları $n = 25$'te kesişir. Grafikteki $n = 1000$ başa baş noktası ayrı bir sayısal örneğe aittir.
- 100 000 elemanlı örnekte doğrusal algoritmanın süresi "2 saniyeden az" diye söylendi. Tabloya göre doğrusu 100 saniyedir, yani yaklaşık 2 dakikadan az.
- $n = 10\,000$ için verilen süre tahminleri (doğrusal "saniyeler", n^2 "dakikalar") abartılı. Saniyede yaklaşık 10^9 basit işlem yapan bir işlemcide 10^4 işlem mikrosaniyeler, 10^8 işlem yaklaşık 0,1 saniye sürer. Önemli olan sıralamadır.
- 80 milyon kişilik listede ikili arama $\lceil \log_2(8 \cdot 10^7) \rceil = 27$ adım sürer, "22-23" değil.
- Hoca Big Omega'yı "en iyi durum", Big Theta'yı "ortalama durum" ile eşleştirdi. Bunlar farklı kavramlardır:
  - O, Ω ve Θ bir fonksiyonun üst, alt ve iki taraflı sınırlarıdır.
  - En kötü, en iyi ve ortalama durum ise hangi girdinin incelendiğini belirtir.
  - Her durumun maliyeti O, Ω ya da Θ ile ifade edilebilir. Örneğin insertion sort'un en kötü durumu $\Theta(n^2)$'dir.
- Küçük o'nun tanımı yalnız eşitliği kaldırmak değildir: $f(n) = o(g(n))$, her pozitif k için bir n_0 bulunabilmesi demektir.
- Slayttaki insertion sort sözde kodunda koşul `j > 0` yazıyor. Dizi 0'dan başladığı için doğrusu `j >= 0`'dır, yoksa ilk eleman hiç karşılaştırılmaz. Yukarıdaki kodda düzeltildi.
- Insertion sort'un ortalama durumda yaklaşık $n^2/4$ karşılaştırma ve $n^2/4$ kaydırma yaptığı bilinir. Hoca $n^2/2$ dedi. Karmaşıklık yine $O(n^2)$'dir.
- Sıralı aramanın ortalama formülü slaytta $(\sum_{i=1}^{n} n)/n$ yazıyor. Doğrusu $(\sum_{i=1}^{n} i)/n = (n+1)/2$'dir.
- Bağlantı problemindeki dizi çözümü literatürde quick-find (union-find'ın bir sürümü) olarak bilinir. Sorgu $O(1)$'dir, ama her birleştirme bütün diziyi taradığı için $O(n)$'dir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.